# The `ic.pipeline` oracle set-up: a cache-blocked pair table, |F| = 1,024 to 16,384

2026-10-20 · study `research/ic_oracle_setup_blocked_20261020` · protocol frozen with the code in `e59331c96` before data · sessions `ECBS1h9e263e582e51` (A, N ≤ 8,192) and `ECBS1h90cd7a0dc213` (B re-run, N = 16,384) · curve `icv1-fp32-t7319-90da43e9` ($p = 2^{32} - 5$, prime order r = 4294959973) · reference `rho.negation`.

## Question

The previous study found the oracle's $|F|^2$ pair-table set-up to be 84–89% of an `ic.pipeline` run's wall time at $|F| \geq 8{,}192$. How much set-up time does a cache-blocked builder (`mitm-prime-blocked`) save against the batched (`mitm-prime-batch`) and scalar (`mitm:negation_folded=1`) builders, everything else unchanged? All three build the same negation-folded table; every arm uses `prime-abscissa:size=N` and `linalg=incremental-gauss` (target-pinned). Session A ran 8 targets × 5 rounds (+1 warm-up); session B ran 4 targets × 2 rounds (+1 warm-up). All 720 analysed records verified.

## Method

The batched builder's arithmetic (one Montgomery batched inversion per abscissa row, x only) costs about 20 ns per sum here; inserting each sum into one hash map that outgrows the caches cost the rest. The blocked builder keeps the arithmetic and its order. For $p < 2^{32}$ and at most $2^{16}$ signed points it packs key and witness into one 64-bit word, gathers each sum by a second hash of its key into one of $|F|^2/2^{14}$ regions, then fills each region, an open-addressed table at most three-quarters full, in one pass. The first pair to reach a key keeps it, as in every other builder. A unit test checks both of its layouts against the batched table on every key of a 20-bit field.

## Result

**S does not change.** 456 of 456 executions (warm-ups included) are identical across the three builders at every N: the recovered logarithm, `total_gae`, every phase's counted operations and counters. Across the §5a band (w = 0, 0.1 and 1 GAE per unpriced operation, every arm alike) the three tie (`ecbench band`).

| N | scalar | batched | blocked | blocked / batched | blocked / scalar | whole run vs scalar | peak RSS batched → blocked |
|---|---|---|---|---|---|---|---|
| 1,024 | 216 ms | 117 ms | 55 ms | 0.47 [0.32, 0.79] | 0.25 [0.15, 0.43] | 0.91 | 0.09 → 0.08 GiB |
| 2,048 | 1.44 s | 739 ms | 238 ms | 0.32 [0.19, 0.44] | 0.17 [0.13, 0.22] | 0.38 | 0.19 → 0.12 GiB |
| 4,096 | 8.40 s | 3.82 s | 1.04 s | 0.27 [0.23, 0.32] | 0.12 [0.10, 0.15] | 0.16 | 0.63 → 0.34 GiB |
| 8,192 | 43.6 s | 20.5 s | 5.29 s | 0.26 [0.24, 0.31] | 0.12 [0.11, 0.15] | 0.13 | 2.37 → 1.25 GiB |
| 16,384 | 224 s | 117 s | 28.2 s | 0.24 [0.22, 0.26] | 0.13 [0.12, 0.14] | 0.13 | 9.15 → 4.65 GiB |

Set-up wall means per run; ratios are Σ over (target, round) pairs, 40 per N (8 at 16,384), with the per-pair range. A/A (`blocked-4096-aa`): identical counts, set-up ratio 1.02 (pair range 0.70–1.46). From N = 2,048 up every per-pair ratio lies below the A/A range. **Every wall figure is descriptive**: most IC runs earned L1 (hypervisor steal), and no comparison had the five admitted pairs ecbench needs.

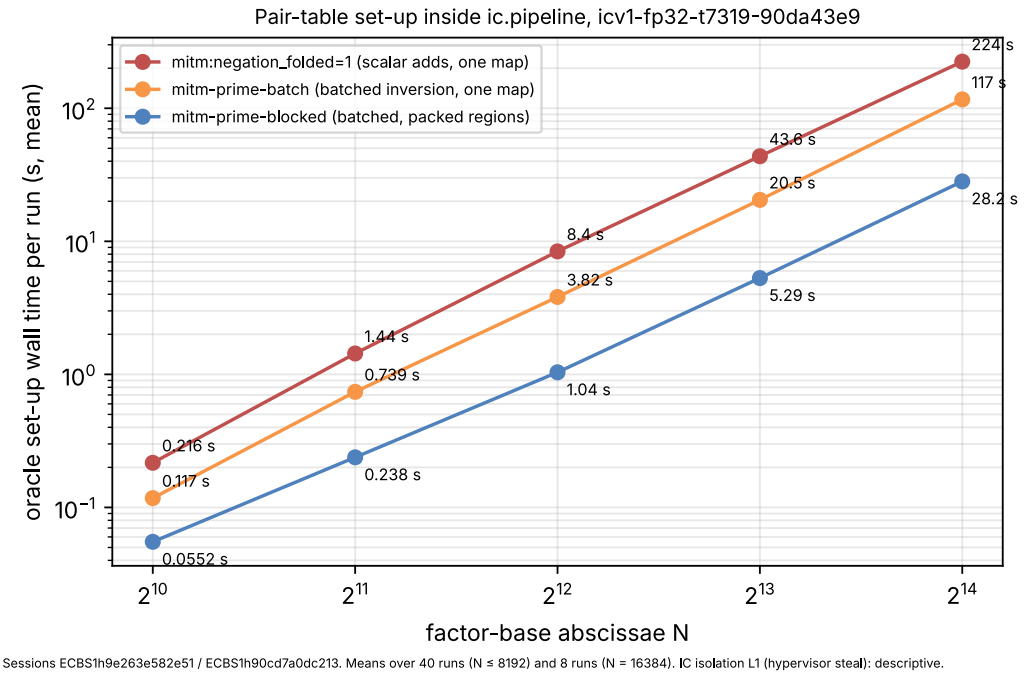



Figure 1: Oracle set-up wall time per run against N (means; descriptive, L1).

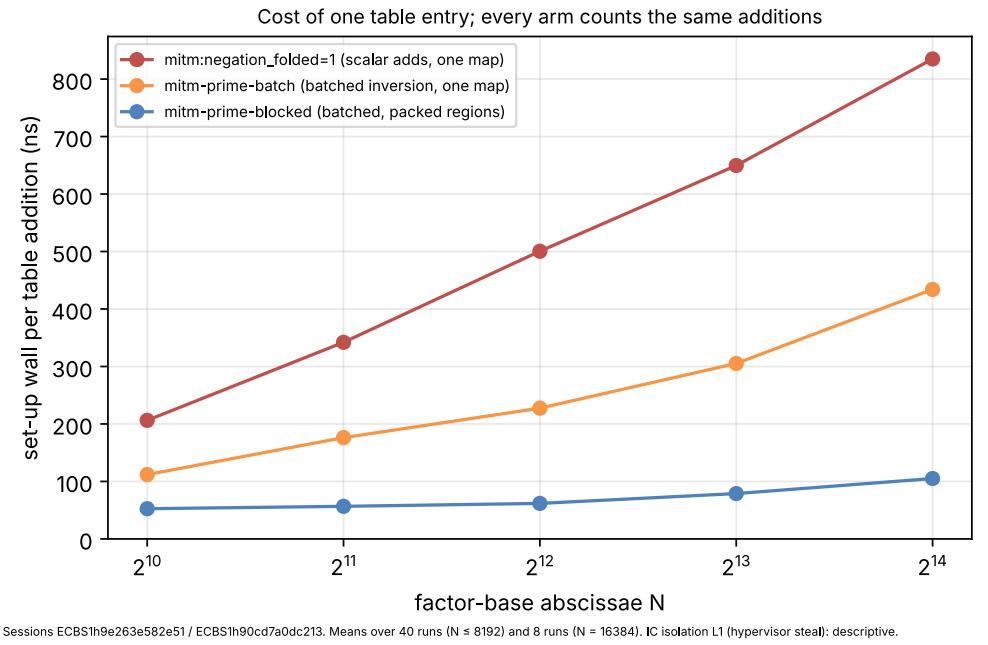



Figure 2: Set-up wall per counted table addition. The blocked builder stays at 53–105 ns as the table grows from 1M to 268M entries.

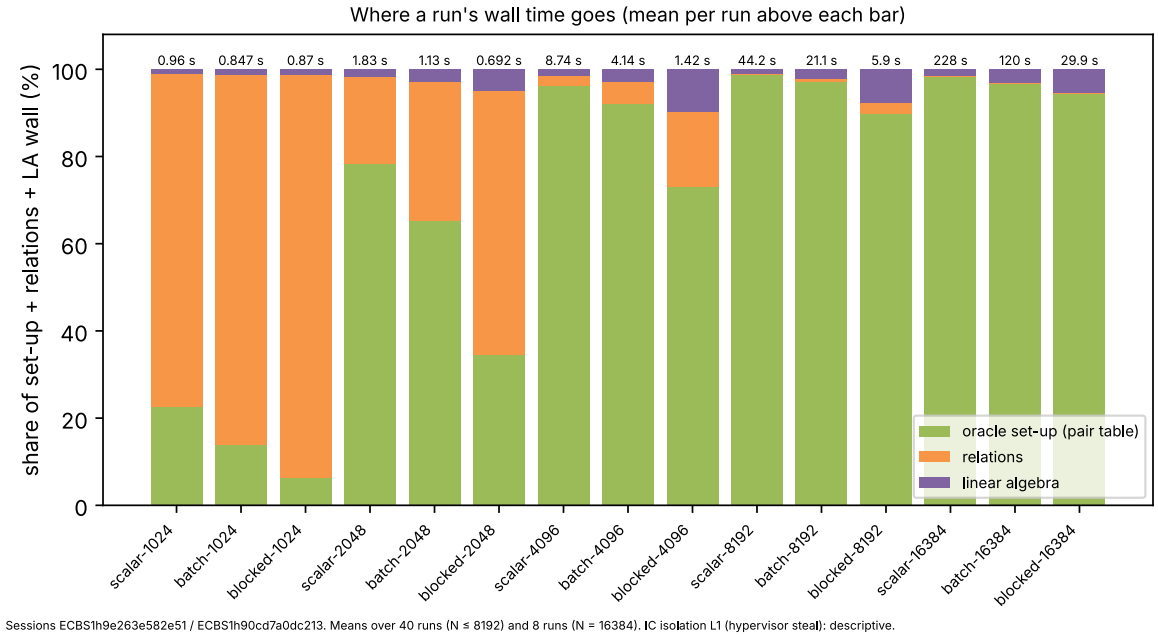



Figure 3: Each phase's share of a run's wall time per arm, with the mean run time above each bar (descriptive).

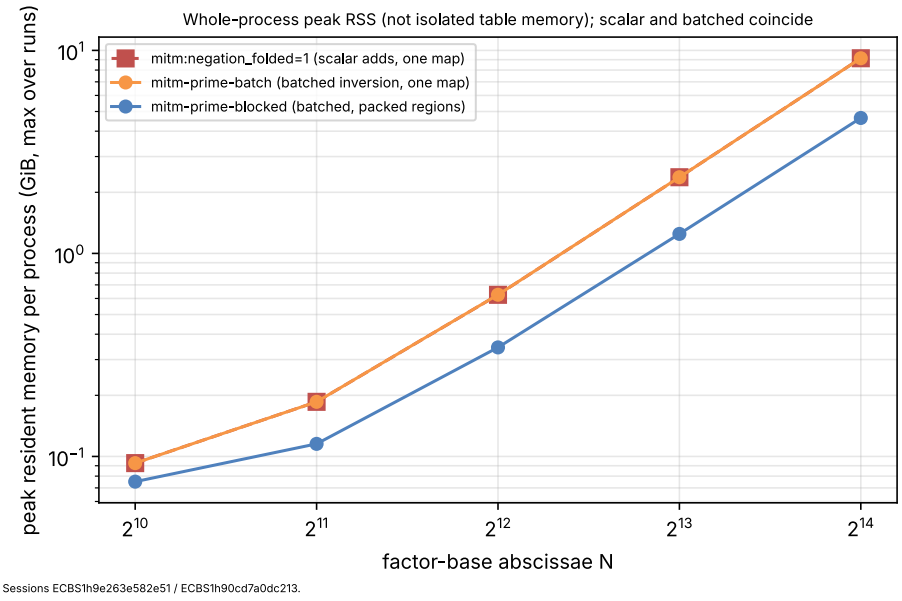



Figure 4: Whole-process peak RSS per arm (maximum over runs).

## Distance remaining to rho

`ops` ratio $S_{IC}$ / $S_{rho}$ (cluster 95% intervals); identical for the three builders.

| **N** | 1,024 | 2,048 | 4,096 | 8,192 | 16,384 |
|---|---|---|---|---|---|
| **w = 0** | 33.0 [27.7, 40.6] | 74.3 [63.7, 90.3] | 269 [231, 326] | 1,062 [912, 1,287] | 5,110 [3,374, 9,867] |
| **w = 1** | 33.0 | 45.4 | 139 | 535 | 2,567 |

The smallest gap is at N = 1,024, where the table is half of S. It has not moved. These gaps are smaller than the previous study's (best 149×) only because that study used the full-rank stop rule (about 4.75 relations per column); this one uses the default target-pinned rule (about 0.5 per column), which the leaderboard's prime ladder already uses. That is the baseline, not an improvement.

## Classification

Engineering, a component (phase) speed-up (AGENTS.md §3). The pair table builds 2–4× faster than batched and 4–8× faster than scalar, at half the memory, on identical output. Predictions P1–P6 held (P2 and P3 descriptively). The set-up's counted cost is already at its floor for this method (one charged addition per key, with the negation fold), so a

smaller S needs fewer charged operations, not a faster table. Deviations: session B was interrupted by an operator time limit at run 27 of 48 and re-run in full from the same spec and binary (the interrupted session is kept); the replay counts were not fixed by the protocol (24/672 and 6/48).